

Лабораторная работа №13-14. Коллекции, обобщения и функциональный стиль в Kotlin

Цель работы: изучить работу с обобщёнными типами (Generics), объектами и расширениями в Kotlin, а также освоить использование коллекций и функций высшего порядка для обработки данных, что является базой для разработки современных Android-приложений.

Шаг 1. Создаём проект

1. Откройте **IntelliJ IDEA**.
2. Создайте новый проект: **File** → **New** → **Project**.
3. Выберите настройки:
 - В качестве языка выберите **Kotlin**.
 - Укажите имя проекта:

Kotlin_Advanced_Lab_13_14_FIO (вместо **FIO** укажите вашу фамилию).

- Снимите галочку с **Add sample code**.
- Нажмите **Create**.

Шаг 2. Подключаем Git

Откройте терминал в **IntelliJ IDEA** (**Alt+F12**).

1. Инициализируйте локальный репозиторий:

git init

2. Создайте первый файл и проверьте его создание:

touch README.md

3. Добавьте файлы в индекс (staging area) и снова проверьте его статус:

git add .

4. Создайте коммит:

git commit -m "Initial commit"

Шаг 3. Подключение локального репозитория GitHub

Подготовьте репозиторий на **GitHub**:

- Создайте новый пустой **public**-репозиторий:

Kotlin_Advanced_Lab_13_14_FIO (замените **FIO** на вашу фамилию).

- Без **README**, **.gitignore** и **License**.

Шаг 4. Подключение удалённого репозитория GitHub

Привяжите локальный репозиторий к удалённому:

```
git remote add origin <URL_вашего_репозитория>
```

```
git push -u origin main/master
```

Шаг 5. Создание описания в README.md

1. Откройте файл **README.md** и добавьте в него краткое описание:

```
# Лабораторная работа №13-14
Коллекции, обобщения и функциональный стиль в Kotlin
## Описание
Данная лабораторная работа посвящена изучению продвинутых возможностей языка Kotlin,
которые активно используются при разработке Android-приложений.
В рамках работы рассматриваются:
- обобщённые типы (Generics);
- коллекции Kotlin;
- функции высшего порядка;
- extension-функции;
Все примеры ориентированы на практическое применение и подготовку к разработке
мобильных приложений.
## Как запустить проект
1. Клонировать репозиторий:
```bash
git clone <URL_репозитория>
```
## Автор
[Ваше ФИО]
## Лицензия
Проект создан в учебных целях.
```

2. В терминале добавим файл проекта:

```
git add .
```

3. Создайте коммит:

```
git commit -m "add structure and update README.md"
```

4. Запушьте изменения:

```
git push
```

Шаг 6. Введение Generics

За десятилетия развития программирования были придуманы различные возможности языков, которые помогают писать более качественный и надёжный код.

Среди них:

- уменьшение количества кода при сохранении смысла;
- абстракция для описания сложных идей;

- механизмы, которые предотвращают ошибки со стороны других разработчиков.

Язык Kotlin не является исключением — в нём есть множество инструментов, которые помогают писать более выразительный и безопасный код.

Однако, если вы сталкиваетесь с программированием впервые, такие возможности могут показаться сложными или неочевидными. Хотя они звучат полезно, не всегда сразу понятно, какие задачи они решают и зачем вообще нужны. Скорее всего, вы уже встречали некоторые из этих возможностей при работе с Jetpack Compose и другими библиотеками, даже если пока не до конца понимаете, как они устроены.

Опыт приходит со временем, но в рамках данной лабораторной работы вы познакомитесь с несколькими важными концепциями Kotlin, которые используются при разработке более крупных и сложных приложений.

В этой лабораторной работе мы рассмотрим:

- **Generics (обобщённые типы)** — написание универсального кода, работающего с разными типами данных;
- **Различные виды классов** (data class, enum class);
- **Singleton** и **companion object** — объекты, существующие в единственном экземпляре;
- **Extension-функции и свойства** — расширение возможностей существующих классов без наследования;
- **Scope-функции** — удобные функции для работы с объектами и их контекстом.

Шаг 7. Обобщённые классы (Generics). Универсальный класс вопроса

Рассмотрим пример приложения с онлайн-викториной (quiz). В такой системе могут встречаться разные типы вопросов:

- **Вопрос с пропуском слова** — ответ представлен строкой (String);
- **Вопрос «верно / неверно»** — ответ представлен логическим значением (Boolean);
- **Математическая задача** — ответ представлен числом (Int).

При этом у всех вопросов есть общие характеристики:

- текст вопроса;

- правильный ответ;
- уровень сложности (easy, medium, hard).

7.1. Проблема дублирования кода

Наивное решение — создать отдельный класс для каждого типа вопроса.

Создайте файл **Question.kt**. Напишите в нём код:

```
class FillInTheBlankQuestion(  
    val questionText: String,  
    val answer: String,  
    val difficulty: String  
)  
  
class TrueOrFalseQuestion(  
    val questionText: String,  
    val answer: Boolean,  
    val difficulty: String  
)  
  
class NumericQuestion(  
    val questionText: String,  
    val answer: Int,  
    val difficulty: String  
)
```

Проблема: Все три класса полностью повторяют друг друга. Отличается только тип свойства **answer**.

Можно было бы использовать наследование, но:

- родительский класс выглядел бы странно без поля `answer`;
- при добавлении нового типа вопроса код всё равно пришлось бы дублировать;
- наследование здесь не решает саму проблему.

7.2. Решение: Generics (обобщённые типы)

Когда у свойства может быть разный тип данных, но логика класса остаётся одинаковой, в Kotlin используется generic-тип.

```
class class name < generic data type > (  
    properties  
)
```

При объявлении класса предоставляется общий тип данных, поэтому его необходимо определить как часть сигнатуры класса. После названия класса ставится левая угловая скобка (<), за которой следует placeholder-имя для типа данных, и после следует правая угловая скобка (>).

Имя placeholder может быть использовано везде, где вы используете реальный тип данных в классе, например, для свойства.

```
class class name < generic data type > (  
    val property name : generic data type  
)
```

Это идентично любому другому объявлению свойства, за исключением того, что вместо типа данных используется placeholder-имя.

Как ваш класс в конечном итоге узнает, какой тип данных использовать? Тип данных, используемый обобщенным типом, передается в качестве параметра в угловых скобках при создании экземпляра класса.

```
val instance name = class name < generic data type > ( parameters )
```

Generics позволяют:

- описать класс один раз;
- подставлять нужный тип данных при создании объекта;
- избежать дублирования кода;
- повысить читаемость и безопасность типов.

7.3. Создание обобщённого класса Question

Перепишем наш код (удалив весь предыдущий):

```
class Question<T>(  
    val questionText: String,  
    val answer: T,  
    val difficulty: String  
)
```

Обратите внимание:

- **T** — это **плейсхолдер (placeholder)** для типа данных;
- реальный тип (**String**, **Boolean**, **Int** и т.д.) будет задан при создании объекта;
- **T** можно использовать в свойствах и методах класса.

7.4. Использование generic-класса

Создадим в нашем файле функцию **main()** и в ней создадим несколько вопросов с разными типами ответов:

```
fun main() {  
    val question1 = Question<String>(  
        questionText = "Речка спятила с ума — По домам пошла сама. ___",  
        answer = "водопровод",  
        difficulty = "medium"  
    )  
    val question2 = Question<Boolean>(  
        questionText = "Небо зелёное. Правда или ложь",  
        answer = false,  
        difficulty = "easy"  
    )  
    val question3 = Question<Int>(  
        questionText = "Сколько дней между полнолуниями?",  
        answer = 28,  
        difficulty = "hard"  
    )  
    println(question1.answer)  
    println(question2.answer)  
    println(question3.answer)  
}
```

Теперь:

- используется один универсальный класс;

- каждый объект сам определяет тип ответа;
- компилятор Kotlin строго контролирует типы.

7.5. Важные замечания

- Тип **T** указывается в угловых скобках при создании объекта:

Question<String>(...)

- Значение `answer` обязано соответствовать указанному типу.
- Чаще всего generic-типы называют:
 - **T** — **type**;
 - **E** — **element**;
 - **K, V** — **key/value** (для коллекций);

но допускаются и более осмысленные имена.

Сохранение изменений в системе контроля версий

После завершения работы выполните следующие команды в терминале:

git add .

git commit -m "Make a reusable class with generics"

git push

Шаг 8. Использование `enum class` для ограничения возможных значений

В предыдущем шаге для свойства `difficulty` использовался тип `String` и возможные значения:

- **"easy"**
- **"medium"**
- **"hard"**

Хотя такой подход работает, у него есть ряд существенных недостатков:

- Опечатки в строках не обнаруживаются компилятором и могут приводить к ошибкам;
- Если значение изменится (например, `"medium"` → `"average"`), придётся искать и исправлять все вхождения строки;
- Ничто не мешает использовать любое другое строковое значение, не входящее в допустимый набор;
- Код становится сложнее поддерживать при добавлении новых уровней сложности.

8.1. Что такое enum class

Для решения подобных задач в Kotlin используется специальный тип классов — перечисления (**enum class**).

enum class применяется, когда:

- существует ограниченный набор возможных значений;
- использование любых других значений недопустимо;
- требуется строгий контроль на уровне компилятора.

Пример из реального мира — стороны света:

- north
- south
- east
- west

Других допустимых значений быть не должно.

Синтаксис enum-класса

```
enum class enum name {  
    Case 1 , Case 2 , Case 3  
}
```

Каждое возможное значение называется **enum-константой**.

Enum-константы:

- перечисляются внутри фигурных скобок;
- разделяются запятыми;
- по соглашению пишутся **ЗАГЛАВНЫМИ БУКВАМИ**.

8.2. Использование enum-констант

Для обращения к значению перечисления используется оператор точка (.).

```
enum name . case name
```


8.3. Добавление перечисления **Difficulty**

Ниже класса `Question` создайте перечисление **Difficulty**.

```
enum class Difficulty {  
    EASY,  
    MEDIUM,  
    HARD  
}
```

8.4. Обновление класса `Question`

Измените тип свойства **difficulty** с `String` на **Difficulty**:

```
class Question<T>(  
    val questionText: String,  
    val answer: T,  
    val difficulty: Difficulty  
)
```

Теперь компилятор гарантирует, что уровень сложности будет только одним из допустимых значений.

8.5. Обновление создания объектов

При создании вопросов используйте `enum`-константы, а не строки:

```
fun main() {  
    val question1 = Question<String>(  
        questionText = "Речка спятила с ума — По домам пошла сама. ___",  
        answer = "водопровод",  
        Difficulty.MEDIUM  
    )  
    val question2 = Question<Boolean>(  
        questionText = "Небо зелёное. Правда или ложь",  
        answer = false,  
        Difficulty.EASY  
    )  
    val question3 = Question<Int>(  
        questionText = "Сколько дней между полнолуниями?",  
        answer = 28,  
        Difficulty.HARD  
    )  
}
```

8.6. Преимущества использования `enum class`

Использование перечислений даёт следующие плюсы:

- защита от опечаток;
- контроль допустимых значений на этапе компиляции;
- упрощение поддержки и рефакторинга;
- более читаемый и самодокументируемый код;
- активное использование в Android-разработке (состояния, события, режимы UI).

Сохранение изменений в системе контроля версий

После завершения работы выполните следующие команды в терминале:

git add .

git commit -m "Use an enum class"

git push

Шаг 9. Использование data class

Многие классы, с которыми вы уже работали (например, наследники Activity), содержат большое количество логики и методов, выполняющих различные действия. Такие классы не только **хранят данные**, но и **управляют поведением**.

Класс **Question**, напротив, служит исключительно для хранения данных:

- текст вопроса;
- ответ;
- уровень сложности.

Он не содержит логики, выполняющей какие-либо действия. Для таких классов в Kotlin существует специальный механизм — **data class**.

9.1. Что такое data class

data class — это класс, предназначенный для хранения данных.

Если класс объявлен как data class, компилятор Kotlin автоматически генерирует для него следующие методы:

- **equals()**
- **hashCode()** (важен при работе с коллекциями)
- **toString()**
- **componentN()** (для деструктуризации)
- **copy()**

Метод **toString()** используется, например, при вызове **println()**.

Без **data class** Kotlin выводит только имя класса и ссылку на объект.

9.2. Проверка поведения обычного класса

В функции **main()** выведите результат вызова **toString()** для **question1**:

```
println(question1.toString())
```

Запустите программу. Вы увидите примерно такой вывод:

```
Question@1e80bfe8
```

Это стандартное поведение обычного класса.

9.3. Преобразование Question в data class

Добавьте ключевое слово **data** перед объявлением класса **Question**:

```
data class Question<T>(  
    val questionText: Str
```

Запустите программу снова. Теперь вывод будет выглядеть так:

```
Question(questionText=Речка спятила с ума — По домам пошла сама. ____,  
    answer=водопровод, difficulty=MEDIUM)
```

Kotlin автоматически сформировал человекочитаемое представление объекта.

Сохранение изменений в системе контроля версий

После завершения шага выполните команды:

```
git add .
```

```
git commit -m "Use a data class"
```

```
git push
```

Шаг 10. Использование singleton-объекта (object)

Во многих ситуациях необходимо, чтобы у класса существовал только один экземпляр.

Примеры:

- статистика игрока в мобильной игре;
- доступ к аппаратному устройству (динамик, камера);
- работа с удалённым источником данных;
- система аутентификации (один активный пользователь).

Создание нескольких экземпляров таких классов может привести к ошибкам и некорректному поведению приложения.

10.1. Что такое singleton

Singleton — это класс, у которого может существовать только один экземпляр.

В Kotlin для этого используется специальная конструкция `object`.

```
object object name {  
    class body 1  
}
```

Особенности:

- экземпляр создаётся автоматически;
- нельзя создать объект через конструктор;
- доступ осуществляется напрямую по имени.

10.2. Создание singleton-объекта

Создайте объект **StudentProgress**:

```
object StudentProgress {  
}
```

Добавьте в него свойства:

```
object StudentProgress {  
    + {  
        var total: Int = 10  
        var answered: Int = 3  
    }  
}
```

10.3. Доступ к singleton-объекту

Так как объект существует в единственном экземпляре, доступ к его свойствам осуществляется напрямую:

object name . property name

Добавьте в **main()**:

```
println("${StudentProgress.answered} of  
${StudentProgress.total} answered.")
```

Ожидаемый вывод:

```
3 of 10 answered.
```

10.4. Companion object (объект-компаньон)

В Kotlin объекты можно объявлять внутри классов.

Если такой объект помечен как **companion**, его свойства доступны напрямую через имя класса.

companion object

object name

10.5. Создание класса Quiz

В текущем файле создайте класс **Quiz**:

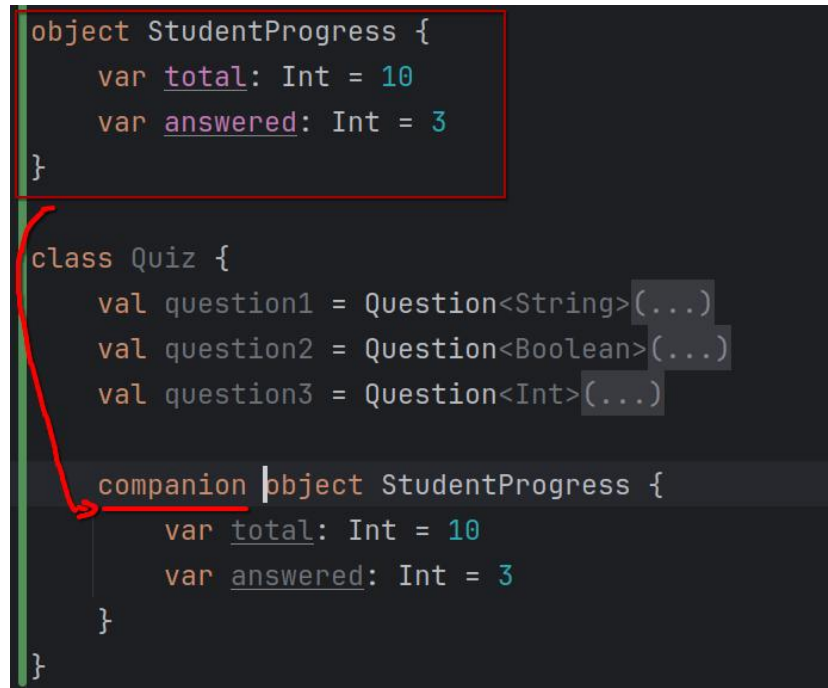
```
class Quiz {  
}
```

Перенесите вопросы из метода **main()** внутрь класса:

```
class Quiz {  
    val question1 = Question<String>(  
        questionText = "Речка спятила с ума — По домам пошла сама. ___",  
        answer = "водопровод",  
        Difficulty.MEDIUM  
    )  
    val question2 = Question<Boolean>(  
        questionText = "Небо зелёное. Правда или ложь",  
        answer = false,  
        Difficulty.EASY  
    )  
    val question3 = Question<Int>(  
        questionText = "Сколько дней между полнолуниями?",  
        answer = 28,  
        Difficulty.HARD  
    )  
}  
  
fun main() {  
    println("${StudentProgress.answered} of ${StudentProgress.total} answered.")  
}
```

10.6. Перенос StudentProgress внутрь Quiz

ПЕРЕНЕСИТЕ объект внутрь класса и пометьте его как companion:



```
object StudentProgress {  
    var total: Int = 10  
    var answered: Int = 3  
}  
  
class Quiz {  
    val question1 = Question<String>(...)  
    val question2 = Question<Boolean>(...)  
    val question3 = Question<Int>(...)  
  
    companion object StudentProgress {  
        var total: Int = 10  
        var answered: Int = 3  
    }  
}
```

10.7. Доступ к companion object

Теперь свойства доступны напрямую через имя класса Quiz. **Измените** метод `main()`:

```
fun main() {  
    println("${Quiz.answered} of ${Quiz.total} answered.")  
}
```

Результат:

```
3 of 10 answered.
```

Сохранение изменений в системе контроля версий

git add .

git commit -m "Use a singleton object"

git push

Шаг 11. Расширение классов с помощью extension-свойств и extension-функций

При работе с Jetpack Compose вы могли заметить необычный, но очень удобный синтаксис.

Например, числовые типы (**Int**, **Double**) внезапно имеют свойства вроде `dp` и `sp`, которые используются для задания размеров элементов интерфейса.

```
padding(16.dp)
```

Возникает логичный **вопрос**: как так получилось, что у встроенных типов **Kotlin** появились свойства для **Android UI**?

Неужели создатели языка заранее знали, что появится Compose?

Конечно, нет!

11.1. Зачем нужны extension-свойства и функции

Когда вы создаёте класс, вы не можете заранее знать:

- как другой разработчик будет использовать ваш код;
- в каком контексте он будет применяться;
- какие дополнительные возможности ему могут понадобиться в будущем.

Добавлять «на всякий случай» лишние методы — плохая практика.

Поэтому Kotlin предоставляет механизм **расширений (extensions)**.

Extensions позволяют:

- добавлять новые свойства и функции к уже существующим типам;
- использовать их через точечную нотацию (`.`), как будто они были частью класса;
- не изменять исходный код класса.

Важно:

- расширения не изменяют сам класс,
- они лишь добавляют удобный синтаксис поверх него.

11.2. Extension-свойства (extension properties)

Чтобы объявить extension-свойство, перед его именем указывается тип, который мы расширяем, и точка:

```
val type name . property name : data type  
    property getter
```

В нашем проекте мы ранее выводили прогресс прохождения теста прямо в `main()`.

Теперь вынесем эту логику в **extension-свойство**.

11.3. Создание extension-свойства `progressText`

Ниже класса `Quiz` объявите extension-свойство для `Quiz.StudentProgress`:

```
val Quiz.StudentProgress.progressText: String
```

Добавьте **getter**, который возвращает строку прогресса:

```
val Quiz.StudentProgress.progressText: String  
+ get() = "${answered} of ${total} answered"
```

Обратите внимание:

- extension-свойства не могут хранить состояние,
- поэтому они всегда являются get-only.

11.4. Использование extension-свойства

Замените код в `main()`:

```
fun main() {  
    println(Quiz.progressText)  
}
```

Запустите программу. **Результат:**

```
3 of 10 answered
```

11.5. Extension-функции (extension functions)

Extension-функции объявляются аналогично — с указанием типа и точки перед именем функции.

```
fun type name . function name ( parameters ) : return type {  
    function body  
}
```

Теперь добавим extension-функцию, которая будет выводить прогресс в виде текстового progress bar.

11.6. Создание extension-функции `printProgressBar()`

Добавьте extension-функцию для `Quiz.StudentProgress`:

```
fun Quiz.StudentProgress.printProgressBar() {  
}
```


Выведите символ  столько раз, сколько вопросов отвечено:

```
fun Quiz.StudentProgress.printProgressBar() {  
    +repeat(times = Quiz.answered) { print("" ) }  
}
```

Выведите символ  для оставшихся вопросов:

```
fun Quiz.StudentProgress.printProgressBar() {  
    repeat(times = Quiz.answered) { print("" ) }  
    +repeat(times = Quiz.total - Quiz.answered) { print("" ) }  
}
```

Перейдите на новую строку и выведите текст прогресса:

```
fun Quiz.StudentProgress.printProgressBar() {  
    repeat(times = Quiz.answered) { print("" ) }  
    repeat(times = Quiz.total - Quiz.answered) { print("" ) }  
    +println()  
    +println(Quiz.progressText)  
}
```

11.7. Использование extension-функции

Обновите **main()**:

```
fun main() {  
    Quiz.printProgressBar()  
}
```

Ожидаемый вывод:

```
  
3 of 10 answered
```

Сохранение изменений в системе контроля версий

После завершения шага выполните:

git add .

git commit -m "Add extension properties and functions"

git push

Шаг 12. Переписываем extension-функции с использованием интерфейсов

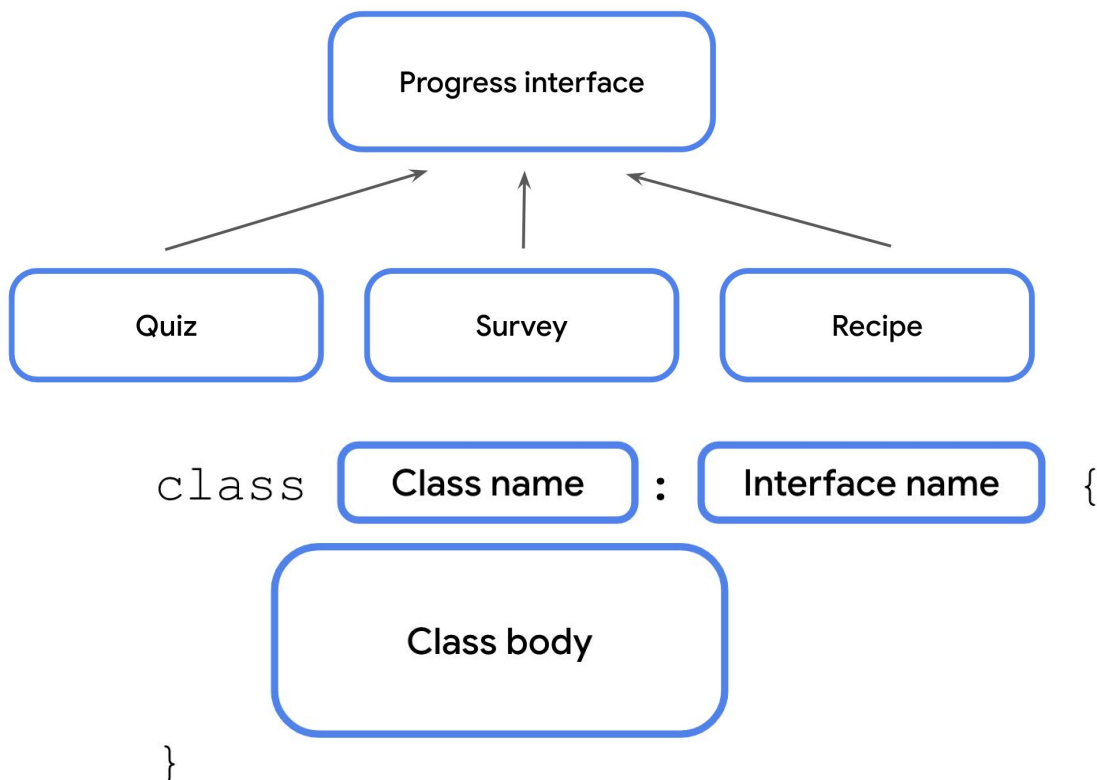
На предыдущем шаге мы добавляли свойства и функции к объекту StudentProgress, не изменяя его напрямую, с помощью extension-свойств и extension-функций.

Это удобный инструмент, но он подходит не всегда.

Иногда:

- у вас есть доступ к исходному коду класса;
- вы заранее знаете, что определённые методы или свойства обязательно должны существовать;
- несколько разных классов должны иметь одинаковый набор функций, но с разной реализацией.
- В таких случаях лучше использовать **интерфейсы (interfaces)**.

Интерфейс — это контракт. Класс, который реализует интерфейс, обязан реализовать все его свойства и методы.



Интерфейс определяется с использованием ключевого слова интерфейса, за которым следует имя в UpperCamelCase, далее открывающие и закрывающиеся

фигурные скобки. В брекетах вы можете определить любые сигнатуры метода или свойства, которые должен реализовать любой класс, соответствующий интерфейсу:

```
interface Interface name {  
    Interface body  
}
```

12.1. Создание интерфейса

Выше класса **Quiz** объявите интерфейс **ProgressPrintable**:

```
interface ProgressPrintable {  
}
```

Добавьте в интерфейс свойство `progressText`:

```
interface ProgressPrintable {  
    +val progressText: String
```

12.2. Реализация интерфейса в классе Quiz

Измените объявление класса **Quiz**, чтобы он реализовывал интерфейс:

```
class Quiz : ProgressPrintable {
```

Добавьте в **Quiz** свойство **progressText**. Так как оно объявлено в интерфейсе, используйте ключевое слово **override**:

```
class Quiz : ProgressPrintable {  
    +override val progressText: String
```

Скопируйте **getter** из ранее созданного **extension-свойства**:

```
class Quiz : ProgressPrintable {  
    override val progressText: String  
    +get() = "${answered} of ${total} answered"
```

Удалите старое extension-свойство:

```
val Quiz.StudentProgress.progressText: String  
    get() = "${answered} of ${total} answered"
```

12.3. Добавление метода в интерфейс

В интерфейсе **ProgressPrintable** объявите метод **printProgressBar()**:

```
interface ProgressPrintable {  
    val progressText: String  
    + fun printProgressBar()  
}
```

Реализуйте этот метод в классе **Quiz**:

```
class Quiz : ProgressPrintable {  
    override val progressText: String  
        get() = "${answered} of ${total} answered"  
  
    + override fun printProgressBar() {  
        repeat(times = Quiz.answered) { print("█") }  
        repeat(times = Quiz.total - Quiz.answered) { print("░") }  
        println()  
        println(progressText)  
    }  
}
```

Удалите extension-функцию:

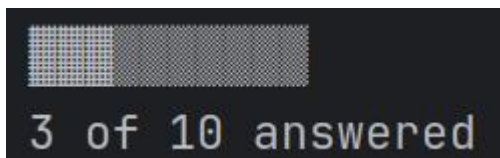
```
fun Quiz.StudentProgress.printProgressBar() {  
    repeat(times = Quiz.answered) { print("█") }  
    repeat(times = Quiz.total - Quiz.answered) { print("░") }  
    println()  
    println(Quiz.progressText)  
}
```

12.4. Обновление main()

Теперь **printProgressBar()** — это метод класса **Quiz**, поэтому нужно создать объект:

```
fun main() {  
    Quiz().printProgressBar()  
}
```

Ожидаемый вывод остаётся тем же:



Интерфейсы широко используются в Kotlin и Android-разработке, например для:

- внедрения зависимостей (Dependency Injection);
- подмены реализаций в тестах (mock-объекты);
- работы с разными платформами (Compose Multiplatform);
- расширения возможностей без изменения исходного кода (например, Modifier в Compose).

Сохранение изменений в системе контроля версий

git add .

git commit -m "Rewrite extensions using interface"

git push

Шаг 13. Scope-functions: доступ к свойствам и методам объекта

Kotlin предоставляет множество инструментов, позволяющих писать более краткий и читаемый код.

Одним из таких инструментов являются **scope-functions**.

Scope-функции позволяют работать со свойствами и методами объекта, не повторяя имя переменной, если это избыточно. Это особенно часто используется в Android-разработке и Jetpack Compose.

Scope-функции — это функции высшего порядка, которые:

- принимают лямбда-выражение;
- временно меняют контекст (scope) внутри этого блока;
- позволяют обращаться к объекту через `it` или `this`.

Рассмотрим на примере.

13.1. Устранение повторений с помощью `let()`

Функция `let()` позволяет обращаться к объекту внутри лямбды через идентификатор `it`.

1. В классе **Quiz** создайте функцию **printQuiz()**:


```
fun printQuiz() {  
}
```

2. Добавьте код, который выводит данные всех вопросов без score-функций:

```
fun printQuiz() {  
    println(question1.questionText)  
    println(question1.answer)  
    println(question1.difficulty)  
    println()  
  
    println(question2.questionText)  
    println(question2.answer)  
    println(question2.difficulty)  
    println()  
  
    println(question3.questionText)  
    println(question3.answer)  
    println(question3.difficulty)  
    println()  
}
```

Здесь имя переменной (**question1**, **question2**, **question3**) повторяется много раз.

3. Оберните работу с каждым вопросом в вызов **let()** и замените имя объекта на **it**:

```
fun printQuiz() {  
    question1.let {  
        println(it.questionText)  
        println(it.answer)  
        println(it.difficulty)  
    }  
    println()  
  
    question2.let {  
        println(it.questionText)  
        println(it.answer)  
        println(it.difficulty)  
    }  
    println()  
  
    question3.let {  
        println(it.questionText)  
        println(it.answer)  
        println(it.difficulty)  
    }  
    println()  
}
```

13.2. Вызов функции из main()

В `main()` создайте объект `Quiz`:

```
fun main() {  
    val quiz = Quiz()  
}
```

Вызовите метод `printQuiz()`:

```
fun main() {  
    val quiz = Quiz()  
    + quiz.printQuiz()  
}
```

При запуске программа выведет:

```
Речка спятила с ума — По домам пошла сама. ___  
водопровод  
MEDIUM  
  
Небо зелёное. Правда или ложь  
false  
EASY  
  
Сколько дней между полнолуниями?  
28  
HARD
```

13.3. Вызов методов без переменной с помощью apply()

Scope-функция `apply()` позволяет:

- вызвать методы объекта;
- настроить его;
- вернуть сам объект.

При этом внутри блока используется `this`, а не `it`.

1. Измените `main()`, используя `apply()`:

```
fun main() {  
    val quiz = Quiz().apply { printQuiz() }  
}
```

2. Так как объект `quiz` больше нигде не используется, переменную можно удалить:


```
fun main() {  
    Quiz().apply { printQuiz() }  
}
```

Метод printQuiz() вызывается **без явного обращения к переменной**.

Scope-функции (**let**, **apply**, **also**, **run**, **with**) очень активно используются в Android-разработке, особенно:

- при инициализации объектов;
- при работе с nullable-типами;
- в Jetpack Compose;
- при конфигурации UI и моделей данных.

Сохранение изменений в системе контроля версий

git add .

git commit -m "Use scope-functions"

git push

Шаг 14. Массивы в Kotlin

Что такое массив?

Массив (Array) — это простейший способ хранить несколько значений одного типа данных.

- массив состоит из элементов (items);
- элементы упорядочены;
- доступ к элементам осуществляется по индексу.

Индекс — это целое число, показывающее позицию элемента в массиве.

В Kotlin (и большинстве языков) используется нумерация с нуля:

- первый элемент → индекс 0
- второй → индекс 1
- и так далее

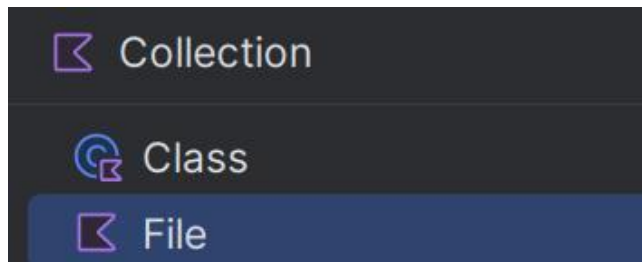
Элементы массива в памяти расположены подряд, поэтому:

- доступ по индексу выполняется быстро (random access);
- размер массива фиксированный — нельзя добавить элемент сверх размера, но можно изменить существующий.

14.1. Создание массива

Массив создаётся с помощью функции **arrayOf()**.

Создайте новый файл в **src** - **Collection.kt**



Создайте в файле **Collection.kt** массив:

```
val rockPlanets = arrayOf<String>("Mercury",  
    "Venus", "Earth", "Mars")
```

Благодаря выводу типов, можно не указывать тип явно:

```
val gasPlanets = arrayOf("Jupiter", "Saturn",  
    "Uranus", "Neptune")
```

Массивы можно объединять оператором **+**:

```
val solarSystem = rockPlanets + gasPlanets
```

14.2. Доступ к элементам массива

Доступ осуществляется по индексу с помощью квадратных скобок. Создайте точку входа **main()** и выведите все планеты Солнечной системы:

```
fun main() {  
    println(solarSystem[0])  
    println(solarSystem[1])  
    println(solarSystem[2])  
    println(solarSystem[3])  
    println(solarSystem[4])  
    println(solarSystem[5])  
    println(solarSystem[6])  
    println(solarSystem[7])  
}
```

14.3. Изменение элемента массива

Значение элемента можно заменить по индексу:

```
fun main() {  
    solarSystem[3] = "Little Earth"  
    println(solarSystem[3])  
}
```

Теперь четвёртый элемент изменён.

14.4. Ограничение размера массива

Попытка выйти за пределы массива приводит к исключению:

```
solarSystem[8] = "Pluto"
```

Ошибка:

```
Exception in thread "main" java.lang  
    .ArrayIndexOutOfBoundsException Create breakpoint :  
    Index 8 out of bounds for length 8
```

Размер массива изменить нельзя.

Если нужен массив большего размера — создаётся новый массив.

```
val newSolarSystem = arrayOf(  
    "Mercury",  
    "Venus",  
    "Earth",  
    "Mars",  
    "Jupiter",  
    "Saturn",  
    "Uranus",  
    "Neptune",  
    "Pluto"  
)
```

Выводим в **main()**

```
fun main() {  
    println(newSolarSystem[8])  
    solarSystem[8] = "Pluto"
```

Массивы — базовая структура данных, но они неудобны, если нужно:

- добавлять и удалять элементы;
- гарантировать уникальность;
- связывать ключи и значения.

Сохранение изменений в системе контроля версий

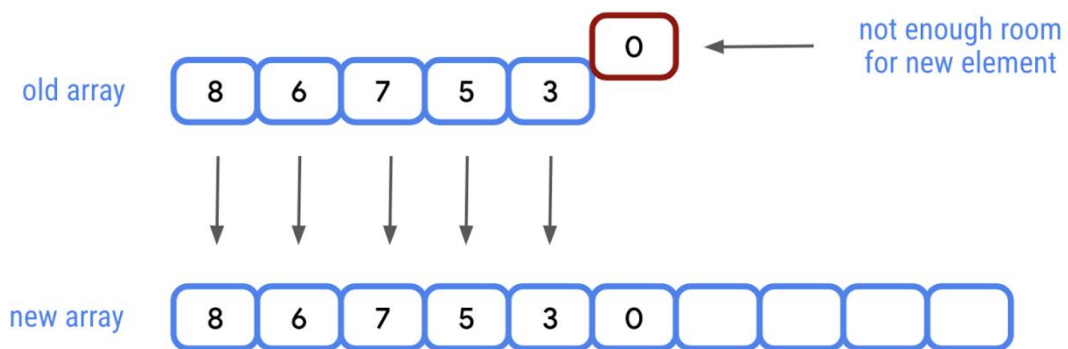
git add .

git commit -m "Arrays in Kotlin"

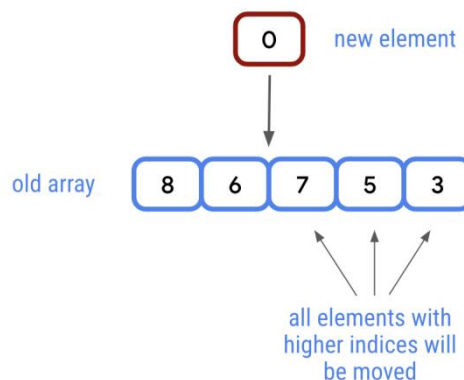
git push

Шаг 15. Lists (Списки)

List — это упорядоченная коллекция с изменяемым размером. Чаще всего она реализована поверх массива, который автоматически увеличивается при необходимости.



- элементы хранятся по порядку;
- можно вставлять элементы в середину списка в определённый индекс;



- добавление и удаление элементов происходит «под капотом» за счёт пересоздания массива.

15.1. List и MutableList

В Kotlin коллекции описываются через **интерфейсы**.

- **List** — интерфейс для только чтения упорядоченной коллекции.
- **MutableList** — расширяет List и добавляет методы для изменения списка (add, remove и т.д.).

Как именно список хранит данные — решает конкретная реализация. В большинстве случаев используется массивная реализация.

15.2. Функция listOf()

Функция **listOf()** создаёт список только для чтения. Обновим **main()**:

```
fun main() {  
    val solarSystem = listOf(  
        "Mercury", "Venus", "Earth", "Mars",  
        "Jupiter", "Saturn", "Uranus", "Neptune"  
    )  
}
```

У списка есть свойство **size**:

```
println(solarSystem.size)
```

Вывод:

```
8
```

15.3. Доступ к элементам списка

Как и у массива, доступ возможен по индексу:

```
println(solarSystem[2])  
println(solarSystem.get(3))
```

Результат:

```
Earth  
Mars
```

Также можно узнать индекс элемента с помощью **indexOf()**:

```
println(solarSystem.indexOf("Earth"))
println(solarSystem.indexOf("Pluto"))
```

Вывод:

```
2
-1
```

Если элемент не найден — возвращается -1.

15.4. Перебор элементов списка (for loop)

Чтобы выполнить действие для каждого элемента списка, используется цикл for:

```
for (planet in solarSystem) {
    println(planet)
}
```

Результат:

```
Mercury
Venus
Earth
Mars
Jupiter
Saturn
Uranus
Neptune
```

Переменная **planet** — временная, соответствует текущему элементу списка.

15.5. Добавление элементов (MutableList)

Чтобы изменять список, нужно использовать **mutableListOf()**:

```
fun main() {
    val solarSystem = mutableListOf(
        "Mercury", "Venus", "Earth", "Mars",
        "Jupiter", "Saturn", "Uranus", "Neptune"
    )
}
```

Добавление в конец списка:

```
solarSystem.add("Pluto")
```

Вставка по индексу:


```
solarSystem.add(index = 3, element = "Theia")
```

15.6. Обновление элементов

Элемент можно заменить по индексу:

```
solarSystem[3] = "Future Moon"  
println(solarSystem[3])  
println(solarSystem[9])
```

15.7. Удаление элементов

Удаление по индексу:

```
solarSystem.removeAt(index = 9)
```

Удаление по значению:

```
solarSystem.remove("Future Moon")
```

Проверка наличия элемента:

```
println(solarSystem.contains("Pluto"))  
println("Future Moon" in solarSystem)
```

Вывод:

```
false  
false
```

Сохранение изменений в системе контроля версий

git add .

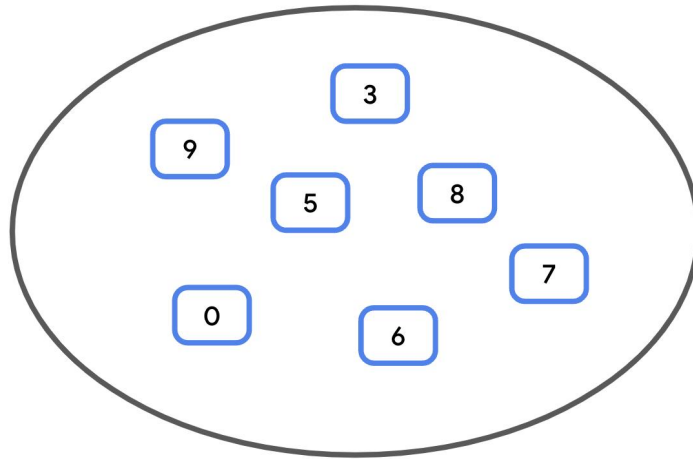
git commit -m "Lists in Kotlin"

git push

Шаг 16. Sets (Множества)

Set — это коллекция без фиксированного порядка, которая **не допускает дубликатов**.

- элементы уникальны;
- порядок элементов не гарантирован;
- нет индексов.



Как Set обеспечивает уникальность

Set опирается на `hashCode()` — целочисленное значение, которое есть у любого объекта в Kotlin.

Даже небольшое изменение объекта (например, одна буква в строке) обычно даёт совершенно другой **hash-код**.

```
"Kotlin".hashCode()    // -204170231
"Kotlin!".hashCode()   // 1131585312
```

На практике **Set** можно представить как:

- внешний массив «корзин» (**buckets**);
- внутри каждой корзины — небольшой список элементов с похожими **hash-кодами**.

Коллизии (одинаковые hash-коды) возможны, но встречаются редко.

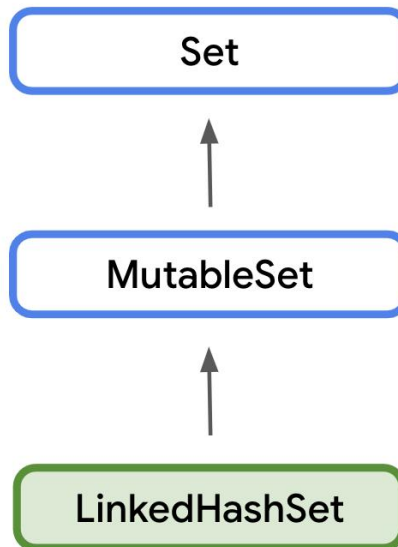
Свойства Set

- Быстрый поиск элемента — значительно быстрее, чем у **List**, особенно на больших объёмах данных.
- Больше памяти, чем **List**, из-за внутренней структуры.
- Гарантия уникальности — главное преимущество **Set**.

Set и MutableSet

- **Set** — только чтение.
- **MutableSet** — позволяет добавлять и удалять элементы.

MutableSet расширяет **Set**, поэтому реализует всё сразу.



16.1. Использование MutableSet

Создадим множество планет с помощью `mutableSetOf()` (по умолчанию используется `LinkedHashSet`). Обновите **main()**:

```
fun main() {  
    val solarSystem = mutableSetOf(  
        "Mercury", "Venus", "Earth", "Mars",  
        "Jupiter", "Saturn", "Uranus", "Neptune"  
    )  
}
```

16.2. Размер множества:

```
println(solarSystem.size)
```

16.3. Добавление элементов

```
solarSystem.add("Pluto")  
println(solarSystem.size)
```

16.4. Проверка наличия элемента:

```
println(solarSystem.contains("Pluto"))
```

Вывод:

```
true
```

Можно использовать и оператор **in**:

```
println("Pluto" in solarSystem)
```

16.5. Дубликаты в Set

Попробуем добавить "Pluto" ещё раз:

```
solarSystem.add("Pluto")  
println(solarSystem.size)
```

Размер не изменится, так как дубликаты запрещены:

```
9
```

16.6. Удаление элементов

Удаление по значению:

```
solarSystem.remove(element = "Pluto")
```

Проверим результат:

```
println(solarSystem.size)  
println(solarSystem.contains("Pluto"))
```

Вывод:

```
8  
false
```

*У Set **нет** индексов, поэтому удалить элемент по позиции невозможно.*

Set отлично подходит, когда важна **уникальность** и **быстрый поиск**.

Сохранение изменений в системе контроля версий

git add .

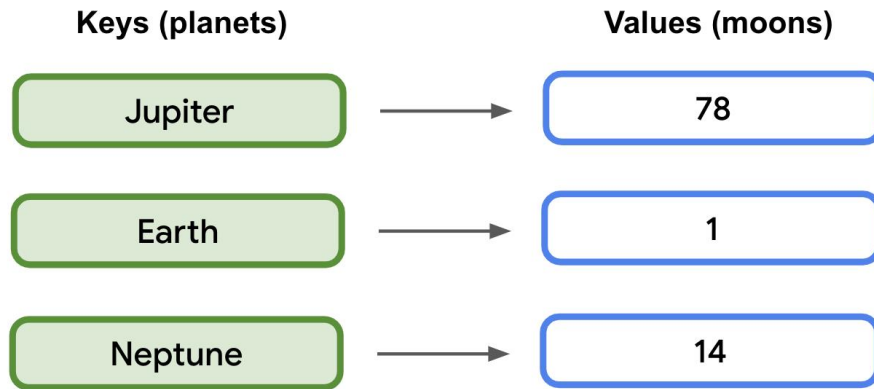
git commit -m "Sets in Kotlin"

git push

Шаг 17. Map collection (Коллекция Map)

Map — это коллекция, состоящая из ключей и значений.

Она называется «**картой**», потому что каждому **уникальному ключу** сопоставляется значение.



Пара **ключ–значение** обычно называется **key-value pair**.

- **Ключи уникальны**
- **Значения могут повторяться**

(например, у "Mercury" и "Venus" по 0 спутников)

Доступ к значению по ключу, как правило, быстрее, чем поиск элемента в большом списке.

Объявление Map

Для создания Map используются функции:

- `mapOf()` — неизменяемая карта
- `mutableMapOf()` — изменяемая карта

```
mutableMapOf<key type, value type>()
```

Map использует два generic-типа:

- тип ключа
- тип значения

Ключ и значение задаются через оператор **to**.

```
val map name = mapOf(  
    key to value,  
    key to value,  
    key to value,  
)
```

17.1. Создание MutableMap

Обновим `main()`:

```
fun main() {  
    val solarSystem = mutableMapOf(  
        "Mercury" to 0,  
        "Venus" to 0,  
        "Earth" to 1,  
        "Mars" to 2,  
        "Jupiter" to 79,  
        "Saturn" to 82,  
        "Uranus" to 27,  
        "Neptune" to 14  
    )  
}
```

Количество элементов в Map:

```
println(solarSystem.size)
```

17.2. Добавление элементов

Добавим новую пару ключ–значение с помощью индексного синтаксиса:

```
solarSystem["Pluto"] = 5  
println(solarSystem.size)
```

Получение значения по ключу:

```
println(solarSystem["Pluto"])
```

17.3. Получение значения и null

Если ключ отсутствует, возвращается null:

```
println(solarSystem.get("Theia"))
```

Вывод программы:

```
null
```

17.4. Удаление элементов

Метод `remove()` удаляет пару по ключу и возвращает удалённое значение (или null):

```
solarSystem.remove(key = "Pluto")  
println(solarSystem.size)
```

Результат:

```
8
```

17.5. Обновление значения

Если ключ уже существует, его значение можно изменить:

```
solarSystem["Jupiter"] = 78  
println(solarSystem["Jupiter"])
```

Вывод:

```
78
```

Map удобно использовать, когда данные логически связаны в формате ключ → значение, и нужен быстрый доступ по ключу.

Сохранение изменений в системе контроля версий

git add .

git commit -m "Map collection in Kotlin"

git push

Итак, резюмируем:

- **Arrays** хранят упорядоченные данные того же типа и имеют фиксированный размер.
- **Arrays** используются для реализации многих других типов коллекций.
- **Lists** - это изменяемая, упорядоченная коллекция.
- **Sets** являются неупорядоченными коллекциями и не могут содержать дубликатов.
- **Maps** работают аналогично наборам и хранящимся парам ключей и значений указанного типа.

Шаг 18. Higher-order functions (Функции высокого порядка)

В лабораторной работе №4 мы уже с вами кратко касались функций высокого порядка. В этом шаге мы более подробно разберёмся с ними.

Функции высокого порядка (high order function) - это функции, которые либо принимают функцию в качестве параметра, либо возвращают функцию, либо и то, и другое.

Создайте файл **HigherOrderFunctions.kt**:

 HigherOrderFunctions.kt

18.1. Функция как параметр функции

Чтобы функция могла принимать другую функцию через параметр, этот параметр должен представлять тип функции. Напишем следующий код:

```
fun main() {  
    displayMessage(mes = ::morning)  
    displayMessage(mes = ::evening)  
}  
  
fun displayMessage(mes: () -> Unit){  
    mes()  
}  
  
fun morning(){  
    println("Good Morning")  
}  
  
fun evening(){  
    println("Good Evening")  
}
```

В данном случае функция **displayMessage()** через параметр **mes** принимает функцию типа **() -> Unit**, то есть такую функцию, которая не имеет параметров и ничего не возвращает.

При вызове этой функции мы можем передать этому параметру функцию, которая соответствует этому типу: `displayMessage(::morning)`

Рассмотрим другой пример параметра-функции, которая принимает параметры:

```
fun main() {
    action(n1 = 5, n2 = 3, op = ::sum) // 8
    action(n1 = 5, n2 = 3, op = ::multiply) // 15
    action(n1 = 5, n2 = 3, op = ::subtract) // 2
}

fun action (n1: Int, n2: Int, op: (Int, Int)-> Int){
    val result = op(n1, n2)
    println(result)
}

fun sum(a: Int, b: Int): Int{
    return a + b
}

fun subtract(a: Int, b: Int): Int{
    return a - b
}

fun multiply(a: Int, b: Int): Int{
    return a * b
}
```

Здесь функция **action** принимает три параметра. Первые два параметра - значения типа **Int**. А третий параметр представляет функцию, которая имеет тип **(Int, Int)-> Int**, то есть принимает два числа и возвращает некоторое число.

В самой функции **action** вызываем эту параметр-функцию, передавая ей два числа, и полученный результат выводим на консоль.

При вызове функции `action` мы можем передать для ее третьего параметра конкретную функцию, которая соответствует этому параметру по типу:

```
action(5, 3, ::sum) // 8
action(5, 3, ::multiply) // 15
action(5, 3, ::subtract) // 2
```

18.2. Возвращение функции из функции

В более редких случаях может потребоваться вернуть функцию из другой функции. В этом случае для функции в качестве возвращаемого типа устанавливается тип другой функции. А в теле функции возвращается лямбда выражение. Например:

```
fun selectAction(key: Int): (Int, Int) -> Int{
    // определение возвращаемого результата
    return when(key){
        1 -> ::sum
        2 -> ::subtract
        3 -> ::multiply
        else -> ::empty
    }
}

fun empty (a: Int, b: Int): Int{
    return 0
}

fun sum(a: Int, b: Int): Int{
    return a + b
}

fun subtract(a: Int, b: Int): Int{
    return a - b
}

fun multiply(a: Int, b: Int): Int{
    return a * b
}
```

Обновим вызов **main()**:

```
fun main() {
    val action1 = selectAction(key = 1)
    println(action1(8,5)) // 13

    val action2 = selectAction(key = 2)
    println(action2(8,5)) // 3
}
```

Здесь функция **selectAction** принимает один параметр - **key**, который представляет тип **Int**. В качестве возвращаемого типа у функции указан тип **(Int, Int) -> Int**. То есть **selectAction** будет возвращать некую функцию, которая принимает два параметра типа **Int** и возвращает объект типа **Int**.

В теле функции **selectAction** в зависимости от значения параметра **key** возвращается определенная функция, которая соответствует типу **(Int, Int) -> Int**.

Далее в функции **main** определяется переменная **action1** хранит результат функции **selectAction**. Так как **selectAction()** возвращает функцию, то и переменная **action1** будет хранить эту функцию. Затем через переменную **action1** можно вызвать эту функцию.

Поскольку возвращаемая функция соответствует типу **(Int, Int) -> Int**, то при вызове в **action1** необходимо передать два числа, и соответственно мы можем получить результат и вывести его на консоль.

Сохранение изменений в системе контроля версий

git add .

git commit -m "Higher order functions"

git push

Шаг 19. Higher-order functions с коллекциями

Для коллекций функции высокого порядка особенно важны, потому что они позволяют выполнять типовые операции — перебор, преобразование, фильтрацию, группировку, сортировку — с минимальным количеством кода.

В этом шаге вы познакомитесь с рядом функций для коллекций:

- > **forEach()**
- > **map()**
- > **filter()**
- > **groupBy()**
- > **fold()**
- > **sortedBy()**

А также ещё раз попрактикуетесь в использовании **lambda**-выражений.

19.1. **forEach()** и строковые шаблоны с лямбдами

Создайте файл **HigherOrderFunctionsList.kt**:

HigherOrderFunctionsList.kt

Добавьте класс **Cookie**. Каждый объект этого класса — один пункт меню.

```
class Cookie(  
    val name: String,  
    val softBaked: Boolean,  
    val hasFilling: Boolean,  
    val price: Double  
)
```

Мы будем форматировать и выводить это меню разными способами.

Ниже класса **Cookie**, создайте список печенья. Тип автоматически определяется как **List<Cookie>** (код можете скопировать ниже):

```
val cookies = listOf(  
    Cookie(  
        name = "Печенье с шоколадом",  
        softBaked = false,  
        hasFilling = false,  
        price = 1.69  
    ),  
    Cookie(  
        name = "Нуга ореховая",  
        softBaked = true,  
        hasFilling = false,  
        price = 1.49  
    ),  
    Cookie(  
        name = "Ванильный крем",  
        softBaked = false,  
        hasFilling = true,  
        price = 1.59  
    ),  
    Cookie(  
        name = "Шоколадно-арахисовое масло",  
        softBaked = false,  
        hasFilling = true,  
        price = 1.49  
    ),  
    Cookie(  
        name = "Сникерс",  
        softBaked = true,  
        hasFilling = false,  
        price = 1.39  
    ),  
    Cookie(  
        name = "Черничный пирог",  
        softBaked = true,  
        hasFilling = true,  
        price = 1.79  
    ),  
)
```

```
Cookie(  
    name = "Сахар и посыпка",  
    softBaked = false,  
    hasFilling = false,  
    price = 1.39  
)  
)
```

Перебор списка с помощью `forEach()`

Функция `forEach()` выполняет переданную в неё функцию для каждого элемента коллекции.

По смыслу она похожа на **for**-цикл или `repeat()`.

Сигнатура метода: `forEach(action: (T) -> Unit)`

- **T** — тип элементов коллекции
- **лямбда** принимает один параметр
- если **параметр один**, можно использовать стандартное имя **it**

Использование `forEach()`

1. Создайте `main()` и вызовите `forEach()` для списка `cookies`, используя **trailing lambda syntax**:

```
fun main() {  
    cookies.forEach {  
  
    }  
}
```

2. В теле лямбды выведите элемент с помощью `println()`:

```
cookies.forEach {  
    + println("Пункт меню: $it")  
}
```

3. Запустите код. Вы увидите только имя класса и идентификатор объекта:

```
Пункт меню: Cookie@7c3df479  
Пункт меню: Cookie@7106e68e  
Пункт меню: Cookie@7eda2dbb  
Пункт меню: Cookie@6576fe71  
Пункт меню: Cookie@76fb509a  
Пункт меню: Cookie@300ffa5d  
Пункт меню: Cookie@1f17ae12
```


Это происходит потому, что выводится результат `toString()` для обычного класса.

Встраивание выражений в строки

Ранее вы использовали строковые шаблоны с `$`. Однако выражения с точкой (`.`) так работать не будут. Попробуем обратиться к свойству `name`:

```
n: $it.name")
```

Результат будет неожиданным:

```
Пункт меню: Cookie@7c3df479.name  
Пункт меню: Cookie@7106e68e.name
```

Чтобы обратиться к свойствам, методам или выполнить выражение, нужно использовать **фигурные скобки** `{}`.

Правильный вариант:

```
${it.name}")
```

Теперь строковый шаблон содержит выражение, и Kotlin корректно подставляет результат. **Вывод:**

```
Пункт меню: Печенье с шоколадом  
Пункт меню: Нуга ореховая  
Пункт меню: Ванильный крем  
Пункт меню: Шоколадно-арахисовое масло  
Пункт меню: Сникерс  
Пункт меню: Черничный пирог  
Пункт меню: Сахар и посыпка
```

Внутри `${ ... }` можно:

- обращаться к свойствам
- вызывать функции
- выполнять вычисления

Это активно используется при работе с коллекциями и лямбдами.

Сохранение изменений в системе контроля версий

git add .

git commit -m "forEach() function"

git push

19.2. map()

Функция **map()** позволяет трансформировать элементы коллекции в новую коллекцию того же размера.

Например, из **List<Cookie>** можно получить **List<String>**, где хранится только название печенья.

Пример: нужно вывести меню пекарни в виде "**название - цена**". Обновляем **main()**:

```
fun main() {  
    val fullMenu = cookies.map {  
        "${it.name} - ${it.price}"  
    }  
    println("Полное меню:")  
    fullMenu.forEach {  
        println(it)  
    }  
}
```

Получаем вывод:

```
Полное меню:  
Печенье с шоколадом - $1.69  
Нуга ореховая - $1.49  
Ванильный крем - $1.59  
Шоколадно-арахисовое масло - $1.49  
Сникерс - $1.39  
Черничный пирог - $1.79  
Сахар и посыпка - $1.39
```

- Внутри **\${...}** можно использовать любые свойства, методы или вычисления.
- **\$** перед числом — просто символ доллара, чтобы показать цену.

Сохранение изменений в системе контроля версий

git add .

git commit -m "map() function"

git push

19.3. filter()

Функция **filter()** создаёт подмножество коллекции, оставляя только элементы, которые соответствуют условию.

В отличие от **map()**, результат может быть меньше исходного размера. Тип элементов остаётся тем же, что и в оригинальной коллекции.

- Лямбда возвращает Boolean.
- Если true — элемент включается в новый список, если false — нет.

Пример: выделим только **soft-baked cookies**:

```
fun main() {  
    val softBakedMenu = cookies.filter {  
        it.softBaked  
    }  
    println("Мягкое печенье:")  
    softBakedMenu.forEach {  
        println("${it.name} - ${it.price}")  
    }  
}
```

Вывод:

```
Мягкое печенье:  
Нуга ореховая - $1.49  
Сникерс - $1.39  
Черничный пирог - $1.79
```

Сохранение изменений в системе контроля версий

git add .

git commit -m "filter() function"

git push

19.4. groupBy() для коллекций

Функция **groupBy()** позволяет превратить список в **Map**, где:

- **ключ** — результат функции, переданной в groupBy
- **значение** — список элементов, для которых функция вернула этот ключ

Пример с нашими печеньками: сгруппируем их по свойству **softBaked**.

```

fun main() {
    val groupedMenu = cookies.groupBy { it.softBaked }
    val softBakedMenu = groupedMenu[true] ?: emptyList()
    val crunchyMenu = groupedMenu[false] ?: emptyList()
    println("Soft cookies:")
    softBakedMenu.forEach {
        println("${it.name} - ${it.price}")
    }
    println("Crunchy cookies:")
    crunchyMenu.forEach {
        println("${it.name} - ${it.price}")
    }
}

```

Вывод:

```

Soft cookies:
Нуга ореховая - $1.49
Сникерс - $1.39
Черничный пирог - $1.79
Crunchy cookies:
Печенье с шоколадом - $1.69
Ванильный крем - $1.59
Шоколадно-арахисовое масло - $1.49
Сахар и посыпка - $1.39

```

Как это работает:

- > cookies.groupBy { it.softBaked } → возвращает Map<Boolean, List<Cookie>>
- > [true] и [false] дают список печений по типу
- > ?: emptyList() нужен, если для ключа ничего не найдено

Итог: groupBy() удобен для разделения коллекции на категории по значению свойства.

Сохранение изменений в системе контроля версий

git add .

git commit -m "groupBy() function"

git push

19.5. fold()

Функция **fold()** аккумулирует значения коллекции в одно значение. Полезно для суммирования, подсчёта общего количества и т.п.

```
fun main() {  
    val totalPrice = cookies.fold(initial = 0.0) { total, cookie ->  
        total + cookie.price  
    }  
    println("Total price: ${totalPrice}")  
}
```

Объяснение:

- 0.0 — начальное значение аккумулятора (total)
- Лямбда { total, cookie -> total + cookie.price } вызывается для каждого элемента
- Результат каждого вызова передается как total для следующего элемента

Вывод:

```
Total price: $10.83
```

Сохранение изменений в системе контроля версий

git add .

git commit -m "fold() function"

git push

19.6. sortedBy()

Функция **sortedBy()** сортирует коллекцию по значению свойства, указанного в лямбде.

```
fun main() {  
    val alphabeticalMenu = cookies.sortedBy { it.name }  
    println("Меню в алфавитном порядке:")  
    alphabeticalMenu.forEach {  
        println(it.name)  
    }  
}
```

Вывод:

```
Меню в алфавитном порядке:  
Ванильный крем  
Нуга ореховая  
Печенье с шоколадом  
Сахар и посыпка  
Сникерс  
Черничный пирог  
Шоколадно-арахисовое масло
```

Сохранение изменений в системе контроля версий

git add .

git commit -m "sortedBy() function"

git push

Шаг 20. Добавление описания в README.md

В файле **README.md** добавьте описание и примеры тем, что были изучены в данной лабораторной работе.

Сохранение изменений в системе контроля версий

git add .

git commit -m "Refactor README.md"

git push

Самостоятельное задание

Задача: Библиотечная система

Цель задания: спроектировать структуру библиотечной системы в соответствии с заданными требованиями.

Требования к реализации:

1. Generic-класс **Library<T>**

Создайте обобщённый (generic) класс **Library<T>**, предназначенный для хранения коллекции элементов произвольного типа **T**. Внутри класса должна быть коллекция (например, список или множество) для хранения объектов типа **T**.

2. Data-классы информационных единиц

Создайте следующие классы данных с указанием их полей и назначения:

❖ **Book**

Поля:

◆ **title: String** — название книги

- ◆ **author: String** — автор книги
- ◆ **year: Int** — год издания
- ◆ **isbn: String** — уникальный идентификатор книги

❖ Magazine

Поля:

- ◆ **title: String** — название журнала
- ◆ **issue: Int** — номер выпуска
- ◆ **month: String** — месяц выпуска (например, "January", "March")

❖ DVD

Поля:

- ◆ **title: String** — название фильма или видеоматериала
- ◆ **director: String** — режиссёр
- ◆ **duration: Int** — продолжительность в минутах

3. Sealed-класс LibraryItem

Создайте запечатанный (**sealed**) класс **LibraryItem**, который будет использоваться для унифицированного представления всех типов носителей в библиотеке. Он должен включать три подкласса-обёртки:

- **BookItem(book: Book)** — обёртка для книги
- **MagazineItem(magazine: Magazine)** — обёртка для журнала
- **DVDItem(dvd: DVD)** — обёртка для DVD

4. Функции высокого порядка

Создайте следующие функции, укажите их входные параметры, возвращаемые значения и логику работы:

- **filterByYear(...)**...

Возвращает список книг, выпущенных в указанном году.

- **sortByTitle(...)**...

Сортирует список элементов библиотеки по названию (title).

- **groupByAuthor(...)**...

Группирует книги по авторам: ключ — имя автора, значение — список всех книг этого автора.

- **calculateTotalDuration(...)**...

Возвращает общую продолжительность (в минутах) всех DVD в списке.